What You've Done

In this chapter you walked through many of the different layers and tools in Rails, and saw how RSpec provides the ability to test those layers in isolation. You learned about views, controllers, mailers, routing, and background jobs. Focused view tests are possible in Rails but overlap heavily with helper tests, logic placed in presenter objects, and integration testing. Mailers and background jobs can also be tested in isolation. Although each of these layers has testing tools built for them, you may choose to test them via isolation tests or by placing most of the logic in non-Rails Ruby objects and testing those as their own unit.

Testing specific parts of the Rails framework is often tricky. Unlike model testing (which tends to be isolated to the particular model) or integration testing (which covers the entire stack), other layers have boundaries that are more blurred and objects that are harder to isolate for test purposes. Controlling those boundaries is the difference between tests that run quickly and fail only when the logic being tested is incorrect, and tests that are slower and dependent on logic outside the test.

Ideally, request tests are written so that they have minimal interaction with the model. There are costs to be balanced. A request action that has minimal contact with the model and can therefore have that interaction stubbed will often run faster and have fewer points of failure. On the other hand, the stubbing and additional classes that may be needed to mediate a controller-and-model interaction may feel overly complex, especially for boilerplate actions.

I'm aggressive about moving controller logic that interacts with the model to some kind of action object that doesn't have Rails dependencies. The controller logic and controller testing then tends to be limited to correctly dispatching successful and failed actions. That said, many Rails developers, notably including David Heinemeier Hansson, find adding an extra layer of objects to be overkill and think that worry about slow tests is misplaced. I recommend you try both ways and see which one best suits you.

Next up, you'll see the tools the Rails core team uses for testing, and how Minitest and its associated libraries let you test Rails.